

C.R.A.M. CONTROL

Classwork **R**esource & **A**ssessment **M**apping Control

Computer Science 2 | Quarter 1 Project

Akisha Dane A. Gregorio | ak47sha | 8 - Ilang-Ilang

I. Problem Statement

High school students balance 8 to 13 academic subjects simultaneously, each with varying grading weights, difficulty levels, and deadlines. Standard calendar apps and simple to-do lists display *when* tasks are due, but they treat all assignments equally and fail to help students determine *what* to work on first when study time is limited.

This issue directly impacts students who struggle with academic time management. Without a systematic method to map task impact against remaining time, students frequently prioritize easy, low-stakes homework over heavy, high-impact assessments until the last minute. This leads to unorganized study habits, high stress, frantic last-minute cramming, and lower overall grades. Developing an automated, logic-based task prioritizer solves this by dynamically calculating urgency and assessment value, transforming chaotic homework lists into actionable study agendas.

II. Project Objectives

To bridge the gap between complex assignment schedules and daily execution, **C.R.A.M. Control** establishes a structured, logic-based framework to optimize student time management. The following SMART (Specific, Measurable, Attainable, Realistic, Time-bound) objectives define the core functional targets of the system:

1. **Calculate Multi-Subject Priority Scores:** Program an algorithm that *calculates a 1–100 numerical priority score* for active coursework across an 8–13 subject workload based on due dates, grade weights, and required study hours.
2. **Generate Real-Time Sorted Agendas:** Provide an *automatically updated daily agenda* that instantly re-sorts remaining tasks whenever an assignment is added, modified, or marked complete.
3. **Budget Daily Study Time Across the Academic Term:** Maintain an ongoing study-time tracker that *sums required daily hours and alerts users to upcoming deadline clusters* to prevent burnout.

III. Planned Features

To translate these objectives into a functional user experience, **C.R.A.M. Control** integrates specific software capabilities designed to capture task data, execute backend calculations, and present dynamic daily agendas. The following planned features detail the core functionality built into the system to streamline multi-subject task management:

- **Assessment Mapping Input Form:** Allows students to log classwork details, including title, subject, assessment type/weight, due date, and estimated study hours.
- **Automated Priority Score Engine:** Uses mathematical rules to evaluate urgency and grade weight, generating a composite priority score.
- **Sorted Daily Agenda Dashboard:** Displays all active classwork arranged from highest priority score to lowest priority score.
- **Workload & Study Time Budgeter:** Calculates and displays total estimated study time required for the active day to prevent student burnout.
- **Interactive Completion & Status Tracker:** Allows students to mark tasks as "In Progress" or "Completed," triggering automatic agenda re-sorting.

IV. Planned Inputs and Outputs

For these features to operate effectively, the system relies on a structured data flow model that converts raw task information into actionable study schedules. The following inputs and outputs define the specific data points collected from the user and the resulting feedback generated by **C.R.A.M. Control**:

Inputs

- **Task Title** (Text, e.g., "Physics Lab Report #2")
- **Subject Name** (Text, e.g., "General Physics", "Computer Science 2")
- **Assessment Type** (Text selection / input):
 - *Periodic Test / Quarterly Exam* (Highest Priority Weight)
 - *Alternative Assessment (AA) / Major Performance Task*
 - *Major Project / Portfolio*
 - *Long Test*
 - *Laboratory Report / Practical Exam*
 - *Class Presentation / Group Reporting*
 - *Short Quiz / Summative Quiz*
 - *Seatwork / In-Class Activity*
 - *Recitation / Oral Participation*
 - *Homework / Daily Assignment* (Lowest Priority Weight)
 - *Other / General Study*
- **Due Date & Time** (Date/Time format)

- **Estimated Completion Hours** (Numeric, e.g., 2.5 hours)
- **Completion Status** (Option: Not Started, In Progress, Completed)

Outputs

- **Calculated Priority Score** (Numeric score automatically derived from the input Assessment Type, urgency, and estimated duration)
- **Prioritized Task Agenda** (Sorted list displaying critical assignments for all 8–13 subjects at the top)
- **Total Remaining Study Time** (Numeric summary, e.g., "Total estimated study time required today: 3.5 hours")
- **Updated Task Status Confirmation** (Visual checkmark and real-time removal of finished tasks)

V. Logic Plan

```
PROGRAM CRAM_Control
BEGIN

# Step 1: Initialize Global Lookup Map for Weights
DECLARE WeightMap AS DICTIONARY
    WeightMap["periodic test"] = 5.0
    WeightMap["quarterly exam"] = 5.0
    WeightMap["alternative assessment"] = 4.5
    WeightMap["major performance task"] = 4.5
    WeightMap["major project"] = 4.0
    WeightMap["portfolio"] = 4.0
    WeightMap["long test"] = 3.5
    WeightMap["laboratory report"] = 3.0
    WeightMap["practical exam"] = 3.0
    WeightMap["class presentation"] = 2.5
    WeightMap["group reporting"] = 2.5
    WeightMap["short quiz"] = 2.0
    WeightMap["seatwork"] = 1.5
    WeightMap["recitation"] = 1.2
    WeightMap["homework"] = 1.0

# Step 2: Function to Retrieve Assessment Weight
FUNCTION GetAssessmentWeight(assessmentType)
    CONVERT assessmentType TO LOWERCASE

    IF assessmentType EXISTS IN WeightMap THEN
        RETURN WeightMap[assessmentType]
    ELSE
        RETURN 1.0 # Default weight for unlisted tasks
    ENDIF
ENDFUNCTION

# Step 3: Function to Calculate Composite Priority Score
FUNCTION CalculatePriorityScore(daysRemaining, assessmentType,
estimatedHours)
    # 1. Urgency Component
    IF daysRemaining <= 0 THEN
        urgencyScore = 100.0
    ELSE
        urgencyScore = (1.0 / daysRemaining) * 40.0
    ENDIF

    # 2. Weight Component
    weightScore = GetAssessmentWeight(assessmentType) * 10.0
```

```

# 3. Workload Component
workloadScore = estimatedHours * 2.0

# Composite Total
totalScore = urgencyScore + weightScore + workloadScore
RETURN ROUND(totalScore, 2)
ENDFUNCTION

# Step 4: Main Execution Routine
BEGIN
    DECLARE taskList AS ARRAY OF DICTIONARIES
    DECLARE keepAdding AS BOOLEAN = TRUE

    OUTPUT "=== WELCOME TO C.R.A.M. CONTROL ==="

    # User Input Loop
    WHILE keepAdding DO
        INPUT taskTitle
        INPUT subjectName
        INPUT assessmentType
        INPUT daysRemaining
        INPUT estimatedHours

        # Store inputs as a structured record
        CREATE taskRecord AS DICTIONARY
        taskRecord["title"] = taskTitle
        taskRecord["subject"] = subjectName
        taskRecord["type"] = assessmentType
        taskRecord["days"] = daysRemaining
        taskRecord["hours"] = estimatedHours
        taskRecord["status"] = "Pending"

        APPEND taskRecord TO taskList

        INPUT userChoice # Prompt: "Add another task? (yes/no)"
        IF CONVERT_TO_LOWERCASE(userChoice) NOT EQUAL TO "yes"
        THEN
            keepAdding = FALSE
        ENDIF
    ENDWHILE

# Step 5: Process and Filter Active Tasks
DECLARE pendingAgenda AS ARRAY OF DICTIONARIES

FOR EACH task IN taskList DO

```

```

        IF task["status"] NOT EQUAL TO "Completed" THEN
            # Attach dynamic calculated score to record
            task["priorityScore"] =
                CalculatePriorityScore(task["days"],
                    task["type"], task["hours"])
            APPEND task TO pendingAgenda
        ENDIF
    ENDFOR

# Step 6: Sort Agenda Descending by Priority Score
SORT pendingAgenda IN DESCENDING ORDER BY task["priorityScore"]

# Step 7: Display Sorted Output Agenda
    IF LENGTH(pendingAgenda) EQUAL TO 0 THEN
        OUTPUT "No pending assignments! All caught up."
    ELSE
        OUTPUT
        "=====
        OUTPUT
        "                C.R.A.M. CONTROL: PRIORITIZED AGENDA                "
        OUTPUT
        "=====

        DECLARE totalStudyTime AS FLOAT = 0.0

        FOR EACH task IN pendingAgenda DO
            OUTPUT task["priorityScore"], task["subject"],
                task["title"], task["type"], task["days"],
                Task["hours"]
            totalStudyTime = totalStudyTime + task["hours"]
        ENDFOR

        OUTPUT
        "-----"
        OUTPUT
        "TOTAL ESTIMATED STUDY TIME REQUIRED: ", totalStudyTime, " HOURS"
    ENDIF

END PROGRAM

```